

AI ANPR Architecture Roadmap Extension

Milestones 12-15 and Final Delivery Criteria

Version	1.0
Status	Roadmap Extension
Scope	Live ANPR, linked vehicle records, testing, tuning, and final delivery criteria
Project Context	IKH One / CS251 FYP ANPR platform
Prepared Date	23 June 2026

Purpose

This document extends the existing AI ANPR architecture roadmap after Milestone 11. It adds Live ANPR Monitoring as the new Milestone 12, Linked Vehicle Records as the new Milestone 13, rennumbers Testing to Milestone 14, rennumbers Performance and Accuracy Tuning to Milestone 15, and defines revised final delivery criteria.

Confidential project working document

Document Basis

This extension is based on the existing AI ANPR module design and milestone roadmap. The original roadmap already includes Milestone 11 as Realtime RTSP Runtime Architecture. The missing architectural layer is the user-facing live ANPR monitoring experience and the backend/frontend vehicle linkage workflow.

- AI ANPR module design: RTSP/video/image/webcam input, detection, OCR, tracking, voting, evidence, and async backend queue.
- Architecture milestones PDF: foundation through realtime RTSP runtime, then testing and performance tuning.
- Repository inspection focus: AI runtime, Laravel ANPR data model, Laravel migrations, React ANPR monitoring feature, and admin management structure.

Roadmap Renumbering

Original Roadmap Item	Revised Roadmap Item	Reason
Milestone 11 - Realtime RTSP Runtime	Unchanged	AI-side RTSP runtime remains a required production runtime milestone.
Milestone 12 - Testing Architecture	Milestone 14 - Testing Architecture	A new frontend/backend live monitoring milestone is inserted before testing.
Milestone 13 - Performance and Accuracy Tuning	Milestone 15 - Performance and Accuracy Tuning	Tuning should happen after live monitoring and linked vehicle records are complete.
Missing	Milestone 12 - Live ANPR Monitoring Architecture	Covers frontend auto-refresh, live status, new event visibility, and end-to-end live detection display.
Missing	Milestone 13 - Linked Vehicle Record Architecture	Covers automatic vehicle linking/creation and admin vehicle management.

Inspection Summary

Area	Finding	Impact
AI ANPR repo	Live RTSP runtime skeleton exists through RTSP run mode, queue flushing, runtime outputs, and M11 runtime behavior.	Milestone 12 should not duplicate AI RTSP runtime. It should connect AI, backend, and frontend into Live ANPR Monitoring.
AI M11 docs	M11 focuses on RTSP runtime and leaves frontend realtime/polling UI out of scope.	Confirms the gap is frontend live ANPR update, not AI frame processing.

Area	Finding	Impact
Frontend repo	feature/anpr-monitoring exists with views, controller, repository, and datasource structure.	There is a frontend ANPR skeleton that can be extended for live polling and indicator behavior.
Frontend realtime services	Existing realtime services are patrol-oriented.	ANPR should use the same project style but implement ANPR-specific live update behavior.
Backend database	vehicles table supports plate_number, owner_name, vehicle_type, status, source, and notes.	M13 can build on the existing schema.
Backend event relation	anpr_events has nullable vehicle_id and resources can expose vehicle relation data.	The structure already supports linking events to vehicles.
Backend ingestion gap	ANPR event store accepts optional vehicle_id but does not automatically find or create vehicles by plate number.	M13 should add backend-owned automatic vehicle linking and creation.

Milestone 12 - Live ANPR Monitoring Architecture

Goal

Implement the live ANPR monitoring experience after M11 RTSP runtime. This milestone connects the already-working AI RTSP runtime, Laravel ANPR APIs, and React ANPR monitoring page so new detections appear in the frontend without manual refresh.

Scope Clarification

This milestone is not video livestreaming. It is live ANPR event monitoring: new detection rows, evidence availability, status feedback, and visible live update behavior in the React dashboard.

12.1 Existing Skeleton Review

Scope

- Inspect AI RTSP runtime, backend queue flush, event posting, and evidence delivery.
- Inspect Laravel ANPR event list endpoint, sorting, pagination, filters, resources, and auth/permissions.
- Inspect React feature/anpr-monitoring, event list page, controller refresh behavior, repository normalization, and existing realtime service conventions.

Passing Criteria

- Current codebase live-readiness is documented.
- Existing manual refresh behavior is identified.
- Required live-update changes are listed before implementation.

12.2 Backend Latest Event Query Support

Support efficient frontend refresh by adding or confirming query support such as:

```
GET /api/anpr-events?sort=detection_time&direction=desc
GET /api/anpr-events?since=<timestamp>
GET /api/anpr-events?per_page=10
```

Scope

- Minimum: existing list endpoint returns newest detections first.
- Minimum: frontend can safely poll the first page.
- Minimum: no duplicate events are returned in UI state.
- Preferred: since or after_id query returns only newly created detections.
- Preferred: response includes pagination metadata.
- Preferred: response includes normalized event resource with camera, vehicle, and evidence summary.

Passing Criteria

- Latest ANPR events can be fetched reliably.
- New detection appears in API response after AI backend queue posts it.
- Query remains protected by existing auth and permission rules.

12.3 Frontend Live Polling Controller

Scope

- Add liveEnabled state.
- Add lastUpdatedAt state.
- Add polling interval, for example 5 seconds.
- Reuse existing loadEvents behavior.
- Prevent overlapping refresh requests.
- Stop polling on component unmount.
- Keep manual refresh button.
- Reset pagination safely when filters change.

Passing Criteria

- Event list auto-refreshes while page is open.
- Manual refresh still works.
- Filters and pagination remain stable.
- No duplicate rows appear.
- Polling stops when leaving the page.

Suggested controller behavior:

```
page opens
-> load latest events
-> start polling
-> every interval refresh first page
-> merge or replace rows safely
-> update lastUpdatedAt
-> show new detections without manual refresh
```

12.4 Blinking Red LIVE Indicator

Scope

- Add a red blinking dot or chip beside the ANPR table title.
- Display the text LIVE.
- Show tooltip on hover: Live update.
- Show the indicator only when live polling is enabled.
- If polling fails, show a degraded state such as RECONNECTING or a warning tooltip.
- Optionally show the last successful update time.

Passing Criteria

- Red blinking LIVE indicator is visible beside the table title.
- Hover tooltip shows Live update.
- UI does not break on mobile layout.
- Failed refresh state is visible but not noisy.

Suggested placement:

```
ANPR Detections [red blinking dot] LIVE
```

12.5 New Event Highlighting

Scope

- Temporarily highlight new rows.
- Keep highlight short, for example 3 to 5 seconds.
- Do not reorder unexpectedly beyond latest-first ordering.
- Preserve user filters.

Passing Criteria

- New detections are visually noticeable.
- Highlight disappears automatically.
- Existing rows remain stable.

12.6 End-to-End Live ANPR Flow

```
RTSP camera
-> AI detection
-> backend queue
-> Laravel ANPR event/images/logs
-> React ANPR list auto-refresh
-> LIVE indicator visible
```

Scope

- Validate RTSP runtime with backend enabled.
- Confirm backend event, image, and log creation.
- Confirm frontend auto-refresh without browser reload.
- Confirm detail page still displays evidence.
- Confirm camera context remains available.
- Confirm sensitive camera IP is not exposed.

Passing Criteria

- A new RTSP detection appears on the frontend without browser refresh.
- Evidence remains available in detail page.
- Camera context remains available.
- Frontend does not expose sensitive camera IP.
- Backend/API errors are handled gracefully.

Milestone 12 Pass Condition

Live ANPR monitoring works end-to-end: while the AI RTSP runtime is running, new ANPR detections appear in the React ANPR monitoring table automatically, with a blinking red LIVE indicator beside the table title and tooltip text Live update.

Milestone 13 - Linked Vehicle Record Architecture

Goal

Automatically link every ANPR event to a vehicle record. If the detected plate exists in the vehicles table, link the event to that vehicle. If it does not exist, create a new vehicle record with source = auto_detected, then link the event.

Architecture Direction

The backend owns vehicle lookup and creation. The AI module should send plate and detection details; it should not be responsible for deciding which vehicle record owns the event.

13.1 Backend Vehicle Linking Service

Create a dedicated service such as AnprVehicleLinker.

```
normalized plate number
-> find existing vehicle by plate_number
-> if missing, create vehicle
-> return vehicle id
```

Scope

- Normalize plate number before lookup.
- Use a database transaction.
- Use firstOrCreate or equivalent safe logic.
- Do not overwrite existing owner, status, or notes.
- For new records set plate_number to detected plate, source to auto_detected, status to normal, and owner_name/vehicle_type/notes to null.

Passing Criteria

- Existing vehicle is reused.
- Missing vehicle is created once.
- Duplicate vehicle rows cannot be created for the same plate.
- Existing manual vehicle metadata is preserved.

13.2 ANPR Event Store Integration

```
AI posts ANPR event
-> backend validates payload
-> backend normalizes plate
-> backend links or creates vehicle
-> backend creates ANPR event with vehicle_id
-> backend returns event with vehicle relation
```

Scope

- AI does not need to send vehicle_id.
- Backend owns vehicle lookup.
- If AI sends vehicle_id, backend should ignore it or validate it strictly against the detected plate.
- If linked vehicle has status = flagged, event is_flagged should be true.
- If linked vehicle has status = whitelist, event can remain valid but should show whitelist context.

Passing Criteria

- Every new ANPR event has vehicle_id.
- Response includes vehicle data.
- Flagged vehicle status is reflected in event/resource behavior.
- Event creation still works for unknown plates.

13.3 Vehicle API Resource and Endpoints

Expose admin vehicle management APIs:

```
GET    /api/vehicles
GET    /api/vehicles/{vehicle}
PATCH /api/vehicles/{vehicle}
```

Optional later:

```
GET /api/vehicles/{vehicle}/anpr-events
```

Field	Behavior
owner_name	Editable
vehicle_type	Editable
status	Editable: normal, flagged, whitelist
notes	Editable
plate_number	Read-only and immutable
source	System-owned: manual or auto_detected

Scope

- Create or confirm VehicleResource.
- Create list/detail/update controller actions.
- Restrict routes to admin/management permissions.
- Validate only editable fields on update.
- Reject plate number changes at API level.

Passing Criteria

- Admin can list vehicles.
- Admin can view vehicle details.
- Admin can edit owner name, type, status, and notes.
- Plate number is read-only at API level.
- Unauthorized users cannot access management endpoints.

13.4 Frontend Vehicle Management Page

Add vehicle management under admin/management. Recommended frontend module:

```
src/feature/management-vehicle/
```

Suggested pages/components:

```
VehicleList.jsx
VehicleEditDrawer.jsx
VehicleDetail.jsx
VehicleRepository.js
```



```
vehicleManagementService.js  
useVehicleManagementController.js
```

List Table Field	Purpose
Plate number	Primary identifier; read-only in edit flows.
Owner name	Admin-managed owner information.
Vehicle type	Admin-managed type classification.
Status	normal, flagged, or whitelist.
Source	manual or auto_detected.
Notes summary	Operational context.
Created date / last updated date	Audit visibility.
Actions	View or edit.

Scope

- Create vehicle management route under admin/management.
- Render vehicle list table.
- Render edit form or drawer.
- Show plate number as visible but disabled/read-only.
- Allow owner name, vehicle type, status, and notes editing.
- Follow existing project UI conventions.

Passing Criteria

- Admin can open vehicle management page.
- Admin can edit allowed fields.
- Plate number is visible but cannot be changed.
- Save updates backend and refreshes UI.
- Existing project UI conventions are followed.

13.5 ANPR Event Detail Vehicle Link

Scope

- Update ANPR event detail to show linked vehicle information.
- Display plate number, owner, vehicle type, status, notes, and source.
- Add link to vehicle management detail/edit page.
- Handle auto-created unknown vehicles safely.

Passing Criteria

- Event detail clearly shows linked vehicle.
- Admin can navigate from ANPR event to vehicle record.
- Unknown auto-created vehicles are visible and editable later.

Milestone 13 Pass Condition

Every backend ANPR event is linked to a vehicle record. Existing vehicles are reused, unknown vehicles are created automatically, and admins can manage vehicle owner/type/status/notes under admin/management while plate number remains immutable.

Milestone 14 - Testing Architecture

Goal

Protect the completed AI/backend/frontend ANPR architecture from regressions, including live monitoring and linked vehicle records.

14.1 AI ANPR Unit Tests

Scope

- Config validation.
- Source resolution.
- Plate normalization.
- Vote selection.
- Track expiry.
- Duplicate cooldown.
- Backend queue retry.
- Token cache refresh.
- RTSP reconnect counters and shutdown behavior.

Passing Criteria

- All unit tests pass locally.
- No real RTSP camera or real backend is required for unit tests.

14.2 AI ANPR Integration Tests

Required command coverage:

```
python main.py check-config
python main.py run --source image --image samples/images/<sample>.jpg --dry-run --strict
python main.py run --source video --video samples/videos/<sample>.mp4 --dry-run --strict
python main.py flush-backend-queue
```

Scope

- Use a fake backend server where possible.
- Validate image flow.
- Validate video flow.
- Validate backend queue success/failure/retry.
- Validate upload mode or metadata mode behavior.

Passing Criteria

- Image flow creates event, evidence, and summary.
- Video flow creates event, evidence, and summary.
- Backend queue handles success, failure, and retry.
- Evidence mode remains verifiable.

14.3 Laravel Backend Tests

Scope

- ANPR event creation.

- Image upload/metadata creation.
- Event logs.
- Vehicle auto-linking by plate number.
- Vehicle auto-creation for unknown plate.
- No duplicate vehicle for the same plate.
- Plate number immutability.
- Vehicle update permissions.
- ANPR list/latest query.
- Event resource includes vehicle relation.

Passing Criteria

- Existing ANPR tests still pass.
- New vehicle-linking tests pass.
- Unauthorized users are rejected.
- Validation errors are field-level and frontend-readable.

14.4 React Frontend Tests

Scope

- ANPR list renders events.
- Manual refresh works.
- Live polling starts and stops.
- Blinking LIVE indicator appears.
- Tooltip shows Live update.
- New row highlight appears.
- Event detail shows evidence and vehicle.
- Vehicle management page edits allowed fields.
- Plate number input is disabled or read-only.

Passing Criteria

- Component/controller tests pass.
- Repository normalization handles missing vehicle/evidence safely.
- Live polling does not create duplicate rows.

14.5 End-to-End Manual Test Matrix

Manual Test Case	Expected Result
AI dry-run image	Local event/evidence/summary generated.
AI dry-run video	Video finalization generates expected records.
AI RTSP short run	Runtime reads camera and shuts down cleanly.
AI RTSP live backend-enabled run	Backend receives event and evidence.
Backend event creation	ANPR event is stored and retrievable.
Evidence display	Frontend detail displays images.
Frontend live ANPR list	New rows appear without browser refresh.
Vehicle auto-link	Existing plate links to existing vehicle.
Vehicle admin edit	Allowed fields update; plate stays immutable.
Flagged vehicle detection	Event reflects flagged context.

Manual Test Case	Expected Result
Queue retry after backend downtime	Queued job retries and eventually posts.

Scope

- Document each manual test case.
- Capture failed-case logs/screenshots/output.
- Keep final test run reproducible.

Passing Criteria

- All manual test cases are documented.
- Failed cases include useful evidence.
- Final test run is reproducible.

Milestone 14 Pass Condition

AI, backend, frontend, and end-to-end tests cover the complete ANPR flow, including live frontend updates and linked vehicle records.

Milestone 15 - Performance and Accuracy Tuning Architecture

Goal

Tune runtime speed, detection reliability, OCR quality, backend responsiveness, and frontend live update efficiency after correctness is already stable.

Initial Performance Targets

Target processed FPS: 3-5 FPS. Event latency after vehicle disappearance: 2-5 seconds. Backend posting: async and non-blocking. Model loading: once at startup. OCR calls: minimized.

15.1 AI Runtime FPS Tuning

Scope

- Tune ANPR_TARGET_FPS.
- Tune vehicle confidence.
- Tune plate confidence.
- Tune OCR confidence.
- Tune frame skipping.
- Tune device selection.
- Create CPU/GPU profile.

Passing Criteria

- RTSP runtime sustains 3-5 processed FPS on demo hardware.
- Model loads once at startup.
- Backend posting does not block frame processing.

15.2 Detection and OCR Accuracy Tuning

Scope

- Tune plate detection confidence.
- Tune vehicle detection confidence.
- Improve Malaysian plate normalization rules.
- Tune OCR preprocessing.
- Tune minimum vote count.
- Tune early finalization confidence.
- Tune track expiry.

Passing Criteria

- False positives are reduced.
- Valid Malaysian plates are retained.
- OCR noise does not dominate final vote.
- One vehicle normally creates one final event.

15.3 Duplicate and Cooldown Tuning

Scope

- Tune duplicate cooldown.

- Tune track expiry.
- Tune re-entry behavior.
- Tune same-plate repeated detection handling.

Passing Criteria

- Same stationary vehicle does not spam events.
- A genuinely new pass can still create a new event after cooldown.
- Duplicate behavior is documented.

15.4 Backend Query Performance

Scope

- Tune ANPR event list latest-first query.
- Tune plate number search.
- Tune vehicle relation eager loading.
- Tune image/log loading strategy.
- Confirm index usage.

Passing Criteria

- ANPR list page remains responsive with growing event volume.
- Vehicle lookup by plate is indexed and reliable.
- Event detail loads evidence and vehicle data without excessive queries.

15.5 Frontend Live Update Efficiency

Scope

- Tune polling interval.
- Add request cancellation where needed.
- Prevent duplicate rows.
- Tune new event highlight duration.
- Add error backoff.
- Document optional future upgrade to broadcast/SSE/WebSocket.

Passing Criteria

- Live update appears within acceptable delay.
- UI remains smooth.
- Failed polling does not spam backend or user.
- Browser memory usage remains stable during long sessions.

15.6 Deployment Profile Documentation

Scope

- Document CPU profile.
- Document GPU profile.
- Document RTSP camera settings.
- Document backend queue flush interval.
- Document frontend polling interval.
- Document evidence mode.

- Document retention policy.
- Document known accuracy limitations.

Passing Criteria

- Final documentation includes recommended .env values.
- Demo profile and production-like profile are clearly separated.
- Limitations are documented honestly.

Milestone 15 Pass Condition

The platform meets agreed speed, latency, reliability, and usability targets under demo RTSP conditions, with documented tuning values and known limitations.

Revised Final Delivery Criteria

The AI ANPR platform is complete when all of the following criteria are satisfied.

1. AI Runtime Completion

RTSP / Video / Image / Webcam

- > Source Reader
- > Frame Scheduler
- > Vehicle Detection
- > Plate Detection
- > OCR
- > Plate Normalization
- > Vote Buffer
- > Track Finalization
- > Evidence Creation
- > Backend Queue

- Models load once.
- Source opens once per run.
- RTSP reconnects after transient failure.
- Runtime shuts down gracefully.
- Active tracks finalize on shutdown.
- Backend failure does not stop detection.
- events.jsonl, evidence, logs, and summary are generated.

2. Backend Completion

Backend Queue

- > Laravel ANPR Event
- > Laravel ANPR Images
- > Laravel ANPR Logs
- > Linked Vehicle Record

- AI-created events are accepted.
- Evidence is stored or resolvable.
- Event logs are created.
- Vehicle is found by plate number or created automatically.
- Every ANPR event has a linked vehicle_id.
- Plate number is immutable after vehicle creation.
- Backend validation errors are field-level and clear.

3. Frontend Completion

Laravel API

- > React ANPR Monitoring
- > Live Event Table
- > Event Detail
- > Evidence Display
- > Vehicle Link

- ANPR list displays latest detections.
- New detections appear without manual browser refresh.
- Blinking red LIVE indicator appears beside the table title.
- Hover tooltip says Live update.

- New event rows are noticeable.
- Event detail shows evidence, camera, logs, and linked vehicle.
- Sensitive camera IP is not exposed.

4. Vehicle Management Completion

Admin
 -> Management
 -> Vehicles
 -> Edit owner/type/status/notes

- Admin can list vehicles.
- Admin can edit owner name, vehicle type, status, and notes.
- Plate number remains read-only and cannot be changed.
- Auto-detected vehicles can be completed later by admin.
- Flagged/whitelist status is reflected in ANPR event context.

5. Testing Completion

- AI unit/integration/manual tests pass.
- Laravel feature tests pass.
- React controller/component tests pass.
- Live ANPR behavior is tested.
- Vehicle linking and immutability are tested.
- Queue retry and backend downtime behavior are tested.

6. Performance Completion

Metric / Criterion	Target
Target processed FPS	3-5 FPS
Event latency after vehicle disappearance	2-5 seconds
Frontend live update delay	Acceptable for polling; preferably <= 10 seconds
Backend posting	Async and non-blocking
OCR calls	Minimized
Duplicate event spam	Controlled
Deployment and tuning profile	Documented

Final Completion Statement

Final Delivery Ready

The system is final-delivery ready when a vehicle passing an RTSP camera can be detected by the AI runtime, finalized into one ANPR event, delivered with evidence to Laravel, automatically linked to a vehicle record, displayed live in React with a blinking LIVE indicator, and managed by an admin through immutable-plate vehicle records, with tests and performance criteria passed.

Appendix A - Implementation Order Recommendation

1. Finish M11 RTSP runtime acceptance and documentation.

2. Implement M12 backend latest-event query and frontend live polling first.
3. Add the blinking red LIVE indicator and new-row highlight.
4. Implement M13 backend vehicle linker before frontend vehicle management.
5. Add admin vehicle routes, API resource, and frontend management page.
6. Run M14 tests across AI, backend, frontend, and manual end-to-end flows.
7. Complete M15 tuning only after correctness is stable.

Appendix B - Source References

- Project source: ai-anpr-modules.md.
- Project source: AI ANPR Architecture Milestones.pdf.
- Public repository: AI ANPR - <https://github.com/taiga2921/cs251-fyp-anpr>.
- Public repository: Backend - <https://github.com/taiga2921/cs251-fyp-backend>.
- Public repository: Frontend - <https://github.com/taiga2921/cs251-fyp-frontend>.